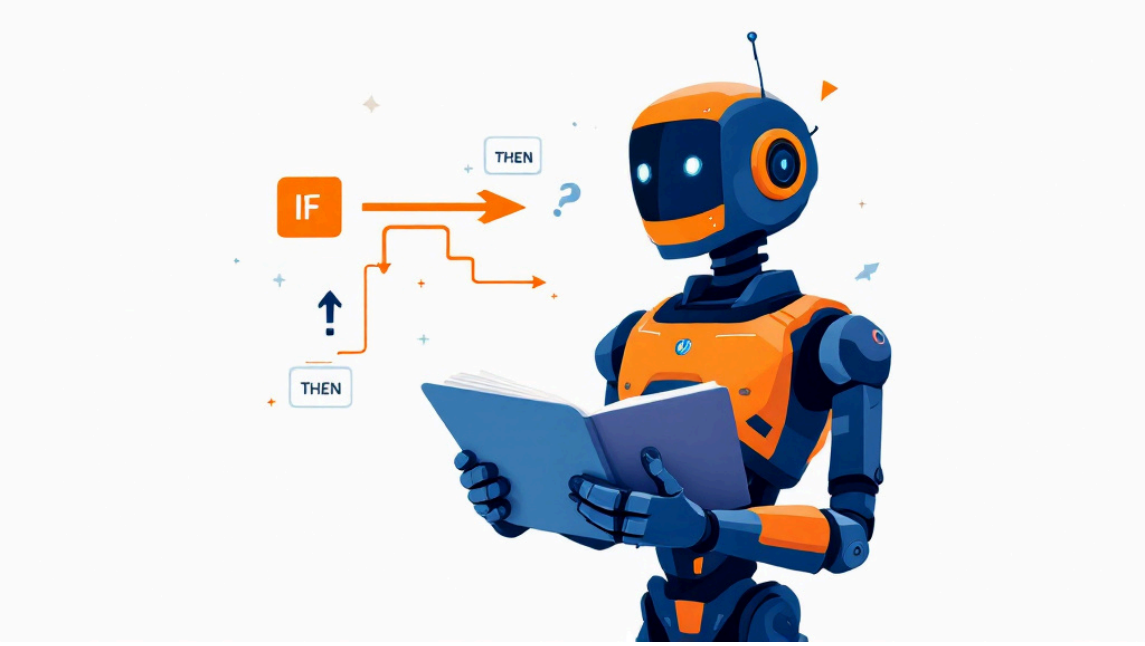
LABORATORIUM 3

Systemy regułowe: reprezentacja i wnioskowanie

Przedmiot	Sztuczna inteligencja
Forma	Laboratorium
Czas	4 h dydaktyczne (190 min), 17.11.2026
Grupy	INLS5_SI
Prowadzący	mgr inż. Szymon Guzik

Cele zajęć

- Zbudować bazę wiedzy z reguł JEŻELI–TO i bazę faktów.
- Zaimplementować cykl wnioskowania w przód z rozstrzyganiem konfliktów.
- Zaimplementować wnioskowanie wstecz z wyjaśnieniem „dlaczego”.
- Obsłużyć niepewność współczynnikami pewności (CF) w stylu systemu MYCIN.
- Pokazać działający moduł projektu (etap 2).



System ekspercki — wiedza specjalisty zapisana jako reguły

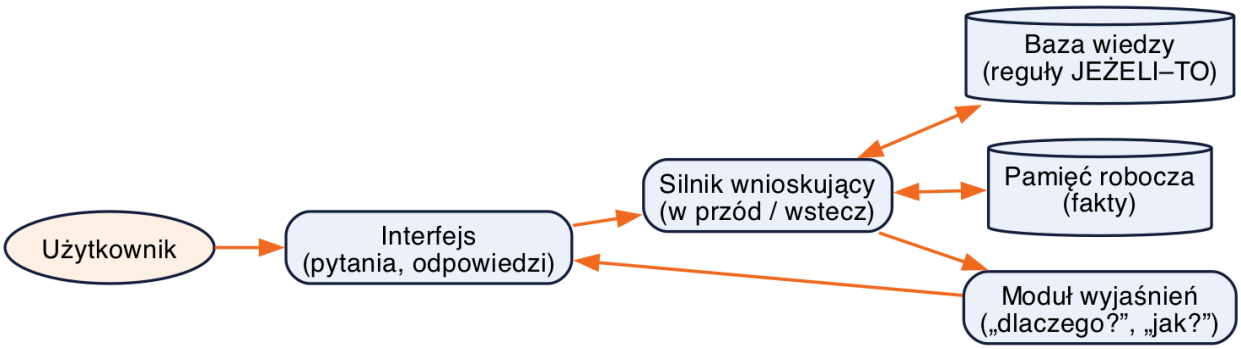
1. Plan zajęć (190 minut)

Czas	Co robimy
0–20 min	Systemy ekspertowe: architektura, reguły produkcji
20–70 min	Wnioskowanie w przód i rozstrzyganie konfliktów — przykład 1, zadania 1–2
70–80 min	Przerwa
80–115 min	Wnioskowanie wstecz i moduł wyjaśnień — zadanie 3
115–150 min	Niepewność: współczynniki pewności — przykład 2, zadanie 4
150–190 min	Projekt — etap 2: demonstracja pierwszego działającego modułu

2. Teoria

System ekspertowy

System ekspertowy (ekspercki) to program, który rozwiązuje problemy z wąskiej dziedziny tak, jak zrobiłby to specjalista: diagnozuje, doradza, klasyfikuje. Wiedzę eksperta zapisujemy jako **reguły produkcji** `JEŻELI warunki TO wniosek`. Kluczowa zasada: **wiedza jest oddzielona od mechanizmu wnioskowania** — nową regułę dopisujemy do bazy bez zmieniania kodu silnika. Klasyczne przykłady to MYCIN (diagnoza zakażeń, lata 70.), XCON (konfiguracja komputerów DEC) czy dzisiejsze systemy scoringu kredytowego.



Architektura systemu ekspertowego

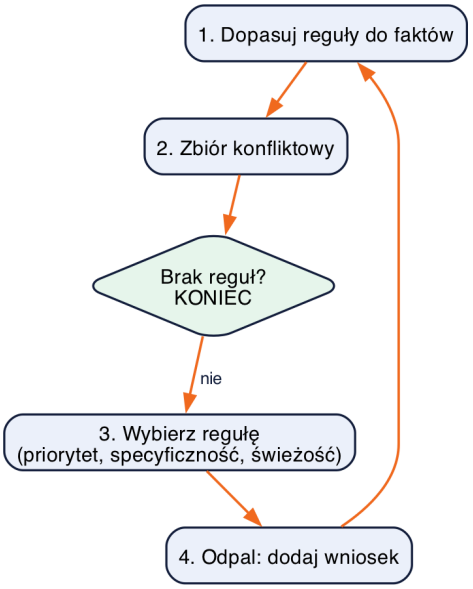
Element	Rola
Baza wiedzy	Reguły — wiedza ogólna, trwała
Pamięć robocza	Fakty o bieżącym przypadku (np. o kliencie)

Element	Rola
Silnik wnioskujący	Dopasowuje reguły do faktów i wyprowadza wnioski
Moduł wyjaśnień	Odpowiada „dlaczego o to pytasz?” i „jak do tego doszedłeś?”
Interfejs	Zadaje pytania, pokazuje wynik

2.1. Wnioskowanie w przód i rozstrzyganie konfliktów

W każdym cyklu silnik znajduje wszystkie reguły, których warunki są spełnione — to **zbiór konfliktowy**. Odpala tylko jedną, wybraną według strategii:

- **priorytet** (waga) nadany przez inżyniera wiedzy,
- **specyficzność** — reguła z większą liczbą warunków jest ważniejsza,
- **świeżość** — pierwszeństwo mają reguły używające najnowszych faktów,
- **refrakcja** — reguła nie odpala dwa razy dla tych samych faktów.



Cykl rozpoznaj–działaj (recognize–act) wnioskowania w przód

3. Przykład 1 — doradca wyboru laptopa

Baza zawiera 6 reguł. Klient programuje, uruchamia maszyny wirtualne, ma budżet powyżej 5000 zł i dużo podróżuje. Program pokazuje kolejne odpalenia (ślad wnioskowania), a potem dowodzi celu „stacja robocza” wnioskowaniem wstecz.

system_ekspercki.py

```python
"""Mały system ekspercki: baza reguł JEŻELI-TO, wnioskowanie w przód z wyjaśnieniem i strategią rozstrzygania konfliktów."""
from dataclasses import dataclass


@dataclass
class Regula:
    nazwa: str
    warunki: frozenset
    wniosek: str
    priorytet: int = 0


BAZA = [
    Regula("R1", frozenset({"budżet_do_3000", "praca_biurowa"}), "laptop_ultrabook_budżetowy"),
    Regula("R2", frozenset({"gry", "budżet_ponad_5000"}), "laptop_gamingowy", priorytet=2),
    Regula("R3", frozenset({"programowanie", "maszyny_wirtualne"}), "potrzeba_32GB_RAM"),
    Regula("R4", frozenset({"potrzeba_32GB_RAM", "budżet_ponad_5000"}), "laptop_stacja_robocza", priorytet=1),
    Regula("R5", frozenset({"dużo_podróży"}), "waga_poniżej_1_5kg"),
    Regula("R6", frozenset({"waga_poniżej_1_5kg", "programowanie"}), "laptop_ultrabook_pro"),
]


def wnioskuj_w_przod(fakty, baza):
    """Cykl: dopasuj reguły -> wybierz jedną (priorytet, potem kolejność) -> odpal -> powtarzaj."""
    fakty, slad = set(fakty), []
    while True:
        gotowe = [r for r in baza if r.warunki <= fakty and r.wniosek not in fakty]   # zbiór konfliktowy
        if not gotowe:
            return fakty, slad
        r = max(gotowe, key=lambda x: (x.priorytet, -baza.index(x)))                # rozstrzygnięcie konfliktu
        fakty.add(r.wniosek)
        slad.append(f"{r.nazwa}: {' ∧ '.join(sorted(r.warunki))} → {r.wniosek}")


def wnioskuj_wstecz(cel, fakty, baza, wciecie=""):
    """Dowodzi celu: fakt znany albo reguła, której wszystkie warunki da się udowodnić."""
    if cel in fakty:
        print(f"{wciecie}✓ {cel} (fakt)")
        return True
    for r in baza:
        if r.wniosek == cel:
            print(f"{wciecie}? {cel} — próbuję {r.nazwa}")
            if all(wnioskuj_wstecz(w, fakty, baza, wciecie + "   ") for w in sorted(r.warunki)):
                print(f"{wciecie}✓ {cel} (z {r.nazwa})")
                return True
    print(f"{wciecie}✗ {cel}")
    return False


if __name__ == "__main__":
    klient = {"programowanie", "maszyny_wirtualne", "budżet_ponad_5000", "dużo_podróży"}
    wynik, slad = wnioskuj_w_przod(klient, BAZA)
    print("WNIOSKOWANIE W PRZÓD")
    for krok in slad:
        print("  ", krok)
    print("rekomendacje:", sorted(f for f in wynik if f.startswith("laptop")))
    print("\nWNIOSKOWANIE WSTECZ: czy laptop_stacja_robocza?")
    wnioskuj_wstecz("laptop_stacja_robocza", klient, BAZA)
```

kod/system_ekspercki.py

```
$ python3 -B kod/system_ekspercki.py
WNIOSKOWANIE W PRZÓD
   R3: maszyny_wirtualne ∧ programowanie → potrzeba_32GB_RAM
   R4: budżet_ponad_5000 ∧ potrzeba_32GB_RAM → laptop_stacja_robocza
   R5: dużo_podróży → waga_poniżej_1_5kg
   R6: programowanie ∧ waga_poniżej_1_5kg → laptop_ultrabook_pro
rekomendacje: ['laptop_stacja_robocza', 'laptop_ultrabook_pro']

WNIOSKOWANIE WSTECZ: czy laptop_stacja_robocza?
? laptop_stacja_robocza — próbuję R4
```

```
   ✓ budżet_ponad_5000 (fakt)
   ? potrzeba_32GB_RAM — próbuję R3
      ✓ maszyny_wirtualne (fakt)
      ✓ programowanie (fakt)
   ✓ potrzeba_32GB_RAM (z R3)
✓ laptop_stacja_robocza (z R4)
```

Zadanie 1 (20 min)

Rozbuduj bazę do co najmniej 12 reguł (np. grafika i montaż wideo, studenci, praca zdalna, system operacyjny). Dodaj regułę **wykluczającą**: jeśli klient potrzebuje wagi poniżej 1,5 kg, system nie może polecić laptopa gamingowego. Jak zapisać negację w tej reprezentacji?

Zadanie 2 (15 min)

Dodaj strategię rozstrzygania **specyficzności** (więcej warunków = pierwszeństwo) i pokaż przypadek, w którym daje inny wynik niż sam priorytet.

Zadanie 3 (30 min)

Zrób interaktywne wnioskowanie wstecz: jeśli faktu brakuje, a żadna reguła go nie wyprowadza, system **pyta użytkownika** (`input()`), zapamiętuje odpowiedź i kontynuuje. Na pytanie „dlaczego?” wypisuje regułę, którą właśnie próbuje udowodnić.

4. Niepewność — współczynniki pewności

Eksperci rzadko są pewni na 100%. System MYCIN wprowadził **współczynnik pewności** CF ∈ [–1, 1]: 1 — pewne, 0 — nic nie wiadomo, –1 — na pewno fałsz. Zasady:

Sytuacja	Wzór
Koniunkcja przesłanek	$CF = \min(CF_1, CF_2, \ldots)$
Odpalenie reguły	CF(wniosek) = CF(przesłanek) × CF(reguły)
Dwa dodatnie świadectwa	$CF = a + b \cdot (1 - a)$
Dwa ujemne	$CF = a + b \cdot (1 + a)$
Świadectwa sprzeczne	CF = (a + b) / (1 – min(

pewnosc.py

```python
"""Współczynniki pewności (CF) w stylu MYCIN: łączenie przesłanek i kilku reguł wspierających ten sam wniosek."""


def cf_przeslanek(cf_list):
    return min(cf_list)                     # koniunkcja przesłanek: najsłabsze ogniwo


def cf_reguly(cf_przeslanek_, cf_reguly_):
    return cf_przeslanek_ * cf_reguly_


def polacz(a, b):
    """Łączenie dwóch niezależnych świadectw za tym samym wnioskiem."""
    if a >= 0 and b >= 0:
        return a + b * (1 - a)
    if a < 0 and b < 0:
        return a + b * (1 + a)
    return (a + b) / (1 - min(abs(a), abs(b)))


objawy = {"gorączka": 0.9, "kaszel": 0.7, "ból_mięśni": 0.6, "katar": 0.2}
reguly = [  # (przesłanki, wniosek, CF reguły)
    (["gorączka", "ból_mięśni"], "grypa", 0.8),
    (["kaszel", "gorączka"], "grypa", 0.5),
    (["katar", "kaszel"], "przeziębienie", 0.7),
    (["katar"], "grypa", -0.3),             # świadectwo przeciwne
]

if __name__ == "__main__":
    wnioski = {}
    for przes, wniosek, cf_r in reguly:
        cf = cf_reguly(cf_przeslanek([objawy[p] for p in przes]), cf_r)
        wnioski[wniosek] = polacz(wnioski[wniosek], cf) if wniosek in wnioski else cf
        print(f"{' ∧ '.join(przes):22} → {wniosek:13} CF = {cf:+.2f}  (łącznie {wnioski[wniosek]:+.2f})")
    print("wynik:", {k: round(v, 2) for k, v in wnioski.items()})

```

kod/pewnosc.py

```
$ python3 -B kod/pewnosc.py
gorączka ∧ ból_mięśni  → grypa         CF = +0.48  (łącznie +0.48)
kaszel ∧ gorączka      → grypa         CF = +0.35  (łącznie +0.66)
katar ∧ kaszel         → przeziębienie CF = +0.14  (łącznie +0.14)
katar                  → grypa         CF = -0.06  (łącznie +0.64)
wynik: {'grypa': 0.64, 'przeziębienie': 0.14}
```

Zadanie 4 (25 min)

Połącz przykład 1 z CF: fakty klienta mają pewność (np. „dużo podróżuję” — 0,6), reguły mają własne CF. Wypisz rekomendacje posortowane według CF i odrzuć te poniżej 0,2.

5. Projekt — etap 2

Demonstracja (40 min)

Każdy zespół uruchamia przy prowadzącym **pierwszy działający moduł** projektu (np. bazę reguł, algorytm genetyczny, wczytywanie danych) i pokazuje historię commitów wszystkich członków zespołu.

6. Literatura

- B. Stefanowicz, *Sztuczna inteligencja: systemy eksperckie. Przewodnik*.
- M. Flasiński, *Wstęp do sztucznej inteligencji*, PWN 2011 — systemy regułowe.
- S. Russell, P. Norvig, *AI: A Modern Approach* — wnioskowanie w logice pierwszego rzędu.